

# Podium: An Open-Source Library for Rendezvous and Docking Guidance, Navigation, and Control with Integrated Formal Verification

Adi Oltean\*

July 2026

## Abstract

Podium is an open-source Python library for developing, testing, and formally validating guidance, navigation, and control (GNC) algorithms for spacecraft rendezvous, proximity operations, and docking (RPOD) in low- and medium-Earth orbit. It provides relative-motion models, convex and sequential-convex trajectory planners, a relative-navigation filter and controllers, a deterministic simulator, and a verification layer. The flight-algorithm core is written in a restricted, statically analyzable Python subset that translates to C, is shown alarm-free on contracted input ranges by the Frama-C Evolved Value Analysis (EVA) plug-in [1], a sound static analyzer, and is compiled with the CompCert verified compiler [2], with the emitted C checked against the Python reference on golden vectors. Several guarantees are re-verified in exact rational arithmetic in the trusted checker: tolerance-free barrier certificates for abort safety, Karush–Kuhn–Tucker (KKT) certificates that bound the suboptimality of an approximate online convex solve within an explicit rational tolerance, and tolerance-free bounds on the global optimum of the nonconvex docking subproblem. This paper describes the architecture, the verification approach, and a reproducible end-to-end example.

## 1 Motivation and significance

Spacecraft rendezvous and docking place strong demands on software. The maneuvers are safety-critical, and some unsafe commands or contact events cannot be corrected after the fact; they run convex and nonconvex optimization in the loop; and the flight code must be shown to be free of runtime errors and faithful to its specification. Mature simulation environments exist, including Basilisk [3], NASA’s 42 [4], Orekit [5], and the General Mission Analysis Tool (GMAT) [6], and Podium does not attempt to replace them for general astrodynamics. Its purpose is narrower: to organize development around spacecraft rendezvous, proximity operations, and docking (RPOD) rather than absolute orbit propagation, and to make formal validation part of ordinary development.

The library is motivated by three needs. Relative motion, approach corridors, keep-out zones, plume impingement, and docking contact are central to RPOD and are a modeling focus distinct from absolute orbit propagation. Convex trajectory optimization is now standard in guidance [7, 8], but a floating-point quadratic program (QP) or second-order cone program (SOCP) solver can return a nominally optimal solution that violates a constraint under rounding, so Podium re-verifies each online solve with an exact-rational Karush–Kuhn–Tucker (KKT) checker, certifying

---

\*Starcloud, Inc. Source and documentation: [github.com/adi-oltean/podium](https://github.com/adi-oltean/podium). Archived at <https://doi.org/10.5281/zenodo.21225268>.

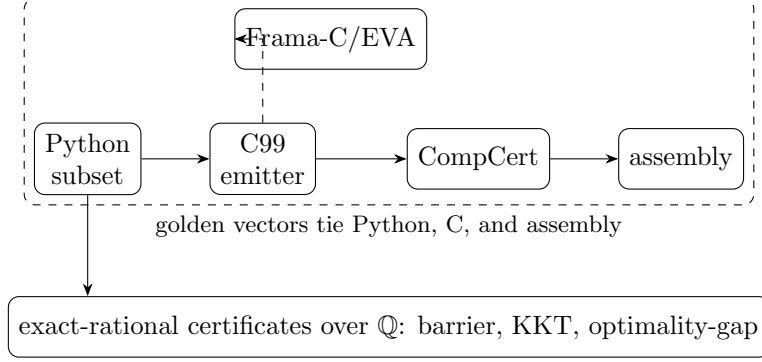


Figure 1: The flight-code path (top): a restricted Python subset is emitted to C99, shown alarm-free by Frama-C/EVA, and compiled by CompCert, with golden vectors tying the Python, C, and assembly on the tested inputs. A separate layer re-runs the exact-rational certificates (verified over  $\mathbb{Q}$ ) together with reachability checks on every change.

the floating-point solution within an explicit rational optimality tolerance, before the command is used. The gap between a Python prototype and analyzable flight-target C, ordinarily crossed by hand, is here crossed mechanically, so that validation evidence applies to the same code path intended for deployment.

## 2 Software description

### 2.1 Architecture

Podium is organized as subpackages under `src/podium`. The `core` package holds the verifiable algorithm core: the Clohessy–Wiltshire dynamics and state-transition matrix, the Yamanaka–Ankersen matrix for elliptic orbits [9], relative orbital elements with Keplerian,  $J_2$ , and  $J_2$ -plus-drag state-transition matrices [10], a quaternion kernel, and fixed-step integrators. The `dynamics` package holds the truth models: nonlinear relative motion with  $J_2$  and differential drag over a seeded stochastic atmosphere, and rigid-body attitude with a suite of environmental torques (gravity gradient, aerodynamic, solar radiation pressure, and magnetic).

The `guidance` package implements convex and sequential-convex trajectory planners on the relative-motion discretizations. The `control` package provides discrete and continuous linear-quadratic regulators, quaternion attitude feedback, and a push-only thruster allocation. The `nav` package implements a relative-navigation extended Kalman filter with a Joseph-form (numerically stable) covariance update and sensor models for relative satellite navigation (relative Global Navigation Satellite System, GNSS), camera, and lidar. The `sim` package provides a deterministic fixed-step engine with bit-identical replays, Signal Temporal Logic (STL) specification oracles, and contact checking. The `verify` and `emit` packages provide the verification layer and the C code generator described below.

### 2.2 A verifiable algorithm core

Flight algorithms are written as pure step functions in a restricted subset of Python: statically shaped arrays, bounded loops, no dynamic allocation, and machine-readable contracts on input ranges and invariants. The subset is chosen so that a sound static analyzer can reason about it and so that it translates line-for-line to C. The `emit` package generates C99 from the subset, annotated

Table 1: Automated verification checks, each re-run in continuous integration. QP: quadratic program. SOCP: second-order cone program. QCQP: quadratically constrained quadratic program. ISA: instruction-set architecture.

| Check                                      | Guarantee                                                                      |
|--------------------------------------------|--------------------------------------------------------------------------------|
| Closed-loop reachability (JuliaReach [12]) | flowpipes avoid unsafe sets on the benchmark models [13]                       |
| Exact-rational barrier certificates        | infinite-horizon abort safety, verified over $\mathbb{Q}$                      |
| Exact-rational KKT certificates            | online QP and SOCP solves certified within an explicit rational optimality gap |
| Exact optimality-gap certificates          | bounds on a nonconvex QCQP optimum, verified over $\mathbb{Q}$                 |
| Sound static analysis (Frama-C/EVA)        | emitted C alarm-free on contracted input ranges                                |
| Verified compiler and cross-ISA            | golden vectors preserved through CompCert and a second ISA                     |
| Independent physics oracle                 | translational truth model cross-validated against Orekit [5]                   |

with ACSL (ANSI/ISO C Specification Language) contracts, together with correctly rounded implementations of the transcendental functions from CORE-MATH [11]. The Frama-C/EVA abstract-interpretation analyzer [1] shows the emitted C free of runtime alarms (division by zero, overflow, invalid access) on the contracted input ranges. The generated C is checked against the Python reference on a suite of golden vectors, bit-exact for arithmetic and square root and correctly rounded for transcendentals; the same vectors are replayed through the CompCert verified compiler [2], which machine-checks semantics preservation from C to assembly for the supported C subset and calling convention, and on a second instruction-set architecture (ISA) to confirm bit-identical results. The golden vectors thus tie the Python reference, the emitted C, and the compiled assembly on the tested inputs.

**Software metadata.** Repository [github.com/adi-oltean/podium](https://github.com/adi-oltean/podium); language Python 3.11+; license MIT; principal dependencies NumPy, CVXPY, and Clarabel, with optional verification tooling (Frama-C, CompCert, JuliaReach). Continuous integration runs on GitHub Actions. The examples and the per-release audit bundle are the primary distribution channel; the library is in active development (v0.x).

### 3 Verification approach

Podium runs verification as a set of automated checks re-executed on every relevant change, rather than as a manual milestone; Table 1 summarizes them.

Two implementation choices support these guarantees. Several certificates are checked in exact rational arithmetic, with no floating point in the trusted checker: a barrier for abort safety, KKT conditions for the online convex solves, and bounds on the global optimum of the nonconvex docking subproblem. The barrier and optimality-gap certificates are tolerance-free, their acceptance being an exact positive-semidefiniteness test together with exact feasibility. The KKT check re-verifies an approximate solver output, so it computes the optimality residuals exactly and accepts the point when they lie within an explicit rational tolerance, which is then a certified bound on the point’s suboptimality. In every case the checker itself introduces no rounding error. The emitted flight C is tied to the Python reference by bit-exact golden vectors and carried to assembly by CompCert, so the validated algorithm and the compiled algorithm agree on the tested inputs. Every tagged release ships an audit bundle, produced by a build script that archives the continuous-integration receipts under a fixed seed, whose hash is reproducible across rebuilds.

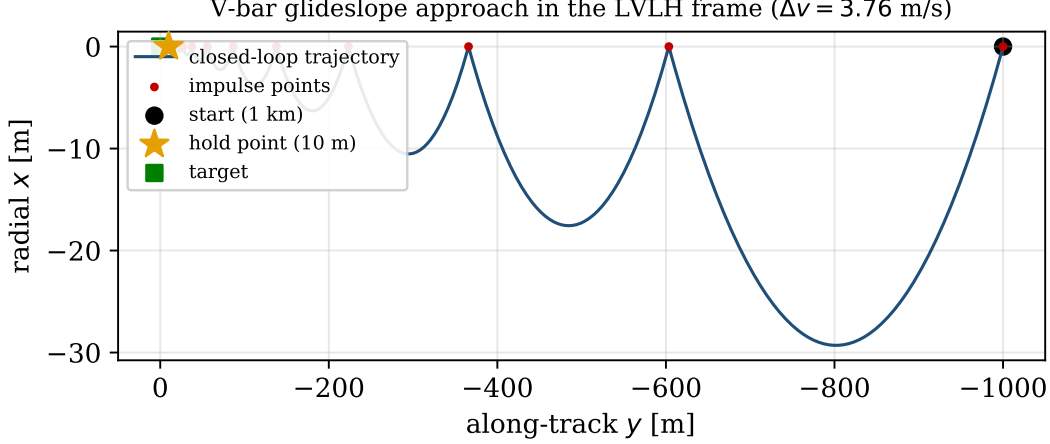


Figure 2: Trajectory of the `vbar_approach.py` demo: a one-kilometer V-bar glideslope approach to a 10 m hold point in the local-vertical/local-horizontal (LVLH) frame, planned on the Clohessy–Wiltshire discretization and propagated under the linear Clohessy–Wiltshire dynamics. The verified pipeline (KKT certification, nonlinear truth model, and barrier) is the reference mission, exercised in continuous integration rather than by this demo.

## 4 Illustrative example

Two artifacts illustrate the library at different depths. A one-command demo, `examples/vbar_approach.py`, plans a one-kilometer approach along the target’s velocity axis (a V-bar approach, in the local-vertical/local-horizontal frame) on the Clohessy–Wiltshire discretization and propagates it under the linear Clohessy–Wiltshire dynamics; Figure 2 shows its trajectory. It is a readable entry point (Python 3.11+, dependencies in `pyproject.toml`, no other setup) and is not the verified pipeline.

The full pipeline is the reference mission (`podium.sim.mission`, exercised by `test.mission.py` in continuous integration): a convex planner whose online solve is re-verified by the exact-rational KKT checker, which certifies the floating-point solution within an explicit rational optimality tolerance; propagation through the nonlinear truth model with  $J_2$  and differential drag; checking of the closed-loop trajectory against approach-corridor and keep-out Signal Temporal Logic specifications; and certification of the passive-drift abort safety (the safe free-drift trajectory after a thruster cutoff) by the exact barrier. An interactive playback of the approach is hosted at <https://adi-oltean.github.io/podium/> as static HTML with no build step, and the emitted flight kernels are exercised in `examples/cfs_nav_app/`, a NASA Core Flight System (cFS) application, with build and run steps in its README.

## 5 Impact and comparison

Each ingredient of Podium is well covered by open prior art; the contribution is their coupling. Table 2 makes this precise against the closest tools on each axis.

On trajectory optimization, the open `SCPToolbox.jl` [14] and `GuSTO.jl` [15] implement the sequential-convex family with rendezvous and free-flyer examples, `ct-scvx` [16] adds continuous-time feasibility with in-house C generation, and `successive_rendezvous` [17] is a released 6-DoF Apollo transposition-and-docking implementation; prong (i) for RPOD is thus established open art, and none of these carries a certificate or verified-code layer. On certificates, `RealCertify` [18], the

Table 2: Podium’s claim is a conjunction: each open tool covers at most one or two prongs. ✓ full, ∼ partial, blank none. Prong (i): a sequential-convex planner; (ii): exact-rational per-solve certificates re-run in continuous integration; (iii): a restricted high-level-source-to-C path checked by golden vectors, Frama-C/EVA, and CompCert. RPOD marks a rendezvous/docking focus.

|                              | (i) SCP | (ii) cert/CI | (iii) verified C | RPOD |
|------------------------------|---------|--------------|------------------|------|
| SCPToolbox.jl [14]           | ✓       |              | ∼                | ∼    |
| GuSTO.jl [15]                | ✓       |              |                  | ∼    |
| ct-scvx [16]                 | ✓       |              | ∼                | ∼    |
| successive_rendezvous [17]   | ✓       |              |                  | ✓    |
| RealCertify [18]             |         | ∼            |                  |      |
| ValidSDP [19]                |         | ∼            |                  |      |
| Verified SDP [20]            |         | ∼            |                  |      |
| Copilot [21]                 |         |              | ✓                |      |
| Credible autocoding [22, 23] | ∼       |              | ∼                |      |
| CVXPYgen [24]                |         |              | ∼                |      |
| Basilisk [3]                 |         |              |                  | ✓    |
| <b>Podium</b>                | ✓       | ✓            | ✓                | ✓    |

Coq-checked ValidSDP [19], and verified SDP [20] produce exact-rational or rigorously bounded certificates for general polynomial and conic problems, but not online, not RPOD-specific, and not re-run in continuous integration. On verified code, the Copilot toolchain [21] couples a restricted high-level source with a C path, formal proof, and CI (for runtime monitors, not guidance); credible autocoding [22, 23] autocodes convex-optimization solvers to contract-annotated C; and CVXPYgen [24] generates C from Python convex problems, none with exact-rational certificates. The simulators Basilisk [3] and NASA’s 42 [4] and the flight-dynamics systems Orekit [5] and GMAT [6] support proximity-operations modeling but no planner, certificate, or verified-code layer.

The novelty is therefore not any single prong but their integration. We are not aware of a publicly released, RPOD-focused library that, in one codebase, couples sequential-convex planners with exact-rational *per-solve* certificates re-run in continuous integration and a restricted Python-to-C flight-code path whose generated C is regression-checked by golden vectors, analyzed by Frama-C/EVA under stated input contracts, and compiled by CompCert over its supported subset. Podium is complementary to the tools above rather than a replacement. It is intended for researchers and engineers developing and validating RPOD guidance and control, and as a reproducible baseline: verification runs on every change and every release carries its evidence, so that results built on the library can be re-checked independently.

## 6 Conclusions

Podium is an open library that organizes guidance, navigation, and control around RPOD scenarios from the outset and treats formal validation as a continuous activity. Its verifiable algorithm core, exact-arithmetic certificates, and verified flight-code path are designed so that validation evidence applies to the same code path intended for deployment, and so that its guarantees can be reproduced by any user. The exact-rational certificates are checked by an  $O(n^3)$  symmetric  $LDL^\top$  factorization, whose factors are themselves the nonnegativity witness; the open problem is to scale this from the per-primitive matrices to the block-banded matrices of a full trajectory, where a fraction-free

elimination exploiting the bandedness would bound the certificate size in the horizon. Development continues toward broader mission coverage and such structure-exploiting exact certificates.

## References

- [1] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- [2] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [3] Kenneth Alcorn, Hanspeter Schaub, and Scott Piggott. Simulating attitude actuation options using the Basilisk astrodynamics software architecture. *Acta Astronautica*, 2020.
- [4] Eric T. Stoneking. 42: A comprehensive general-purpose simulation of attitude and trajectory dynamics and control of multiple spacecraft. NASA Goddard Space Flight Center, open-source software, 2020. <https://github.com/ericstoneking/42>.
- [5] Luc Maisonobe, Véronique Pommier, and Pascal Parraud. Orekit: An open source library for operational flight dynamics applications. In *4th International Conference on Astrodynamics Tools and Techniques (ICATT)*, 2010.
- [6] NASA Goddard Space Flight Center. General mission analysis tool (GMAT). Open-source software, 2020. <https://gmat.gsfc.nasa.gov>.
- [7] Behçet Açıkmeşe and Scott R. Ploen. Convex programming approach to powered descent guidance for Mars landing. *Journal of Guidance, Control, and Dynamics*, 30(5):1353–1366, 2007.
- [8] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, Thomas Lew, Riccardo Bonalli, Marco Pavone, and Behçet Açıkmeşe. Convex optimization for trajectory generation. *IEEE Control Systems Magazine*, 42(5):40–113, 2022.
- [9] Koji Yamanaka and Finn Ankersen. New state transition matrix for relative motion on an arbitrary elliptical orbit. *Journal of Guidance, Control, and Dynamics*, 25(1):60–66, 2002.
- [10] Adam W. Koenig, Tommaso Guffanti, and Simone D’Amico. New state transition matrices for relative motion of spacecraft in perturbed orbits. *Journal of Guidance, Control, and Dynamics*, 40(7):1749–1768, 2017.
- [11] Alexei Sibidanov, Paul Zimmermann, and Stéphane Glondou. The CORE-MATH project. In *29th IEEE Symposium on Computer Arithmetic (ARITH)*, 2022.
- [12] Sergiy Bogomolov, Marcelo Forets, Goran Frehse, Kostiantyn Potomkin, and Christian Schilling. JuliaReach: A toolbox for set-based reachability. In *22nd ACM International Conference on Hybrid Systems: Computation and Control (HSCC)*, 2019.
- [13] Matthias Althoff et al. ARCH-COMP category report: Continuous and hybrid systems with linear continuous dynamics. In *ARCH Workshop*, 2023.
- [14] University of Washington Autonomous Controls Laboratory. SCPToolbox.jl: Sequential convex programming toolbox. Open-source software, 2022. <https://github.com/UW-ACL/SCPToolbox.jl>; companion to [8].

- [15] Riccardo Bonalli, Abhishek Cauligi, Andrew Bylard, and Marco Pavone. GuSTO: Guaranteed sequential trajectory optimization via sequential convex programming. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019. <https://github.com/StanfordASL/GuSTO.jl>.
- [16] Purnanand Elango, Dayou Luo, Abhinav G. Kamath, Samet Uzun, Taewan Kim, and Behçet Açıkmeşe. Continuous-time successive convexification for constrained trajectory optimization. *Automatica*, 180:112464, 2025.
- [17] Danylo Malyuta, Taylor P. Reynolds, Michael Szmuk, and Behçet Açıkmeşe. Six-degree-of-freedom rendezvous trajectory generation via successive convexification with state-triggered constraints. Open-source research code, AIAA SciTech, 2020. [https://github.com/dmalyuta/successive\\_rendezvous](https://github.com/dmalyuta/successive_rendezvous).
- [18] Victor Magron and Mohab Safey El Din. RealCertify: A Maple package for certifying non-negativity. *ACM Communications in Computer Algebra*, 52(2):34–37, 2018.
- [19] Érik Martin-Dorel and Pierre Roux. ValidSDP: A Coq library for verified SDP-based positivity certificates. Coq library, 2017. <https://sourcesup.renater.fr/www/validsdp>.
- [20] Christian Jansson. On verified numerical computations in convex programming. *Japan Journal of Industrial and Applied Mathematics*, 26:337–363, 2009.
- [21] Ryan G. Scott, Mike Dodds, Ivan Perez, Alwyn E. Goodloe, and Robert Dockins. Trustworthy runtime verification via bisimulation (experience report). *Proceedings of the ACM on Programming Languages*, 7(ICFP):305–321, 2023.
- [22] Timothy Wang, Romain Jobredeaux, Heber Herencia-Zapana, Pierre-Loïc Garoche, Arnaud Dieumegard, Eric Feron, and Marc Pantel. From design to implementation: An automated, credible autocoding chain for control systems. In *Advances in Control System Technology for Aerospace Applications*, volume 460 of *Lecture Notes in Control and Information Sciences*. Springer, 2016. arXiv:1307.2641.
- [23] Raphaël Cohen, Eric Feron, and Pierre-Loïc Garoche. Verification and validation of convex optimization algorithms for model predictive control. *Journal of Aerospace Information Systems*, 17(5), 2020. arXiv:2005.12588.
- [24] Maximilian Schaller, Goran Banjac, Steven Diamond, Akshay Agrawal, Bartolomeo Stellato, and Stephen Boyd. Embedded code generation with CVXPY. *IEEE Control Systems Letters*, 6:2653–2658, 2022.